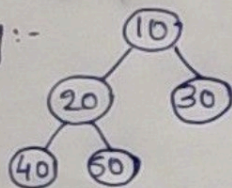


Module - 3

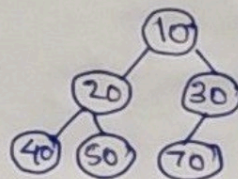
Advanced Heap Structure

Heap is a complete binary tree. where all the levels, except the last level should be completely filled. and all the node should be left justified. (Insertions start from left side).

eg:-



this is complete binary tree



this is not a complete binary tree.

* There are two types of Heap

→ Minimum Heap : ~~minimizing~~

→ Maximum Heap.

↳ Min. Heap

the value of parent node should be less than or equal to either of its children.

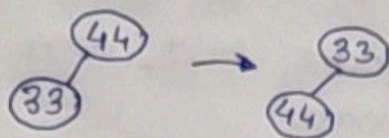
Question : construct min-heap from above question values: 44, 33, 77, 11, 55, 88, 66.

and Inserting 44,

(44)

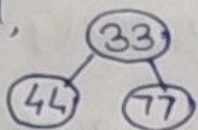
*inserted as root node
*min-heap.

*inserting 33,

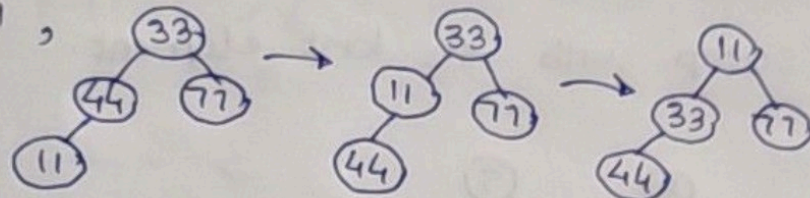


*33 is swapped with 44
*min-heap

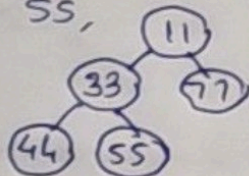
*inserting 77,



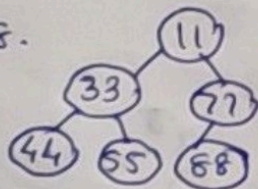
*inserting 11,



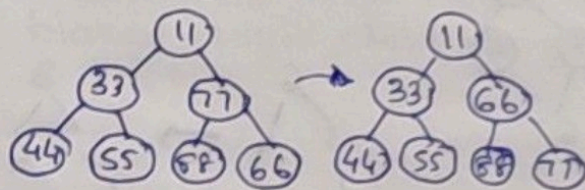
*inserting 55,



*inserting 88.

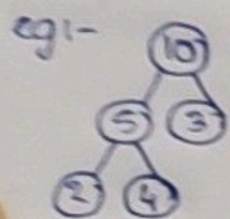


*inserting 66,

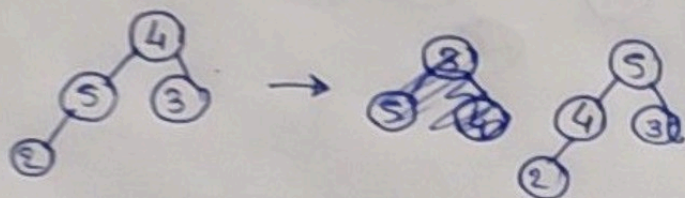


DELETION

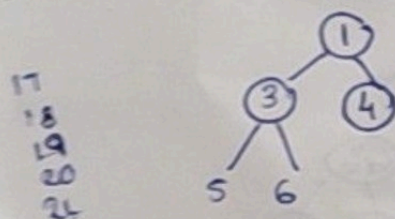
- Step 1: remove root node.
Step 2: move the last element of last level to root.
Step 3: compare the value of this child node with its parent.
Step 4: if value of parent is less than child, then swap them.
in case of maximum-heap.
Step 5: repeat step 3 and 4 until heap property holds.



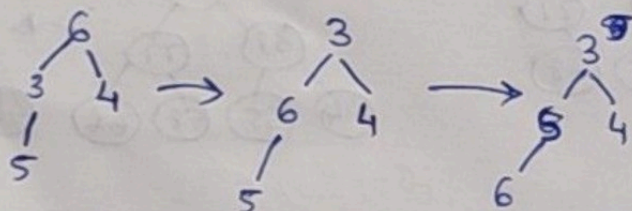
the element to be deleted is 10. (root)
and swap with the last element.



Question 2



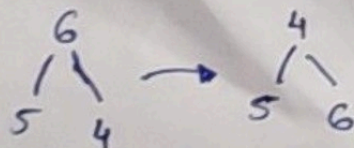
delete the node 5.



this is min heap

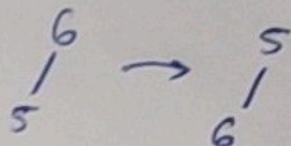
replace with last element
as root node

now step 2:-



now delete 4.

now step 3



then delete 5,
the heap will be

6

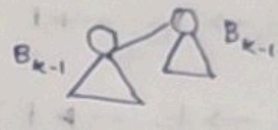
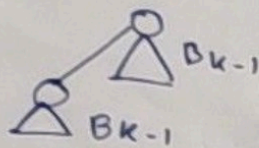
Binomial Heap

Binomial Heap is a collection of Binomial trees

Binomial Tree :-

Binomial tree B_k is an ordered tree defined recursively.
 Binomial tree B_0 consist of single node.
 a binomial tree B_k consist of 2 binomial tree B_{k-1} that are linked together (left biased)

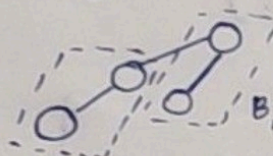
$$B_k \rightarrow 2B_{k-1}$$



$$B_0 \rightarrow 1 \rightarrow \bigcirc$$

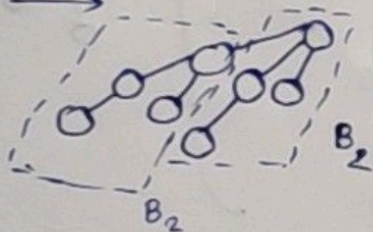
$$B_1 \rightarrow 2B_0 \rightarrow \begin{array}{c} \bigcirc_{B_0} \\ | \\ \bigcirc_{B_0} \end{array}$$

$$B_2 \rightarrow 2B_1 \rightarrow$$

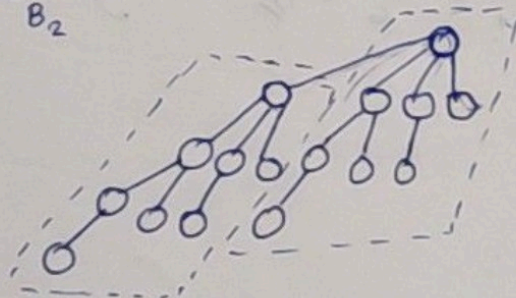


B_1 left biased aayit child aayitt add aayit

$$B_3 \rightarrow 2B_2 \rightarrow$$



$$B_4 \rightarrow 2B_3 \rightarrow$$



Properties of binomial Tree

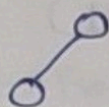
- 1) there are 2^k nodes.
- 2) the height of the tree is k .
- 3) there are exactly kC_i nodes at depth i for $i=0, 1, 2, \dots, k$

$$kC_i = \frac{k!}{i!(k-i)!} = nC_r = \frac{n!}{r!(n-r)!}$$

$$B_0 \rightarrow {}^0C_0 \rightarrow 1$$

$$B_1 \rightarrow {}^1C_0 \rightarrow 1$$

$$\rightarrow {}^1C_1 \rightarrow 1$$

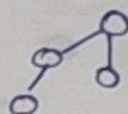


$$B_2 \rightarrow {}^2C_0 \rightarrow 1$$

$$\rightarrow {}^2C_1 \rightarrow \frac{2!}{1!} = 2$$

$$\rightarrow {}^2C_2 \rightarrow \frac{2!}{2! \times (0)!} = \frac{2!}{2} = 1$$

$$\frac{2}{0(2)!} = \frac{2}{2} = 1$$

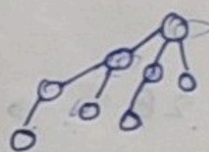


$$B_3 \rightarrow {}^3C_0 \rightarrow 1$$

$$\rightarrow {}^3C_1 \rightarrow \frac{3!}{1! \times (2)!} = \frac{6}{1 \times 2} = \frac{6}{2} = 3$$

$$\rightarrow {}^3C_2 \rightarrow \frac{3!}{2! \times (3-2)!} = \frac{6}{2 \times 1} = 3$$

$$\rightarrow {}^3C_3 \rightarrow \frac{3!}{3! \times 0!} = \frac{6}{6} = 1$$



$$B_4 \rightarrow {}^4C_0 \rightarrow 1$$

$$\rightarrow {}^4C_1 \rightarrow$$

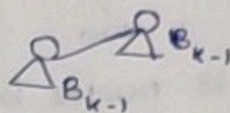
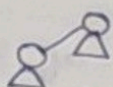
Binomial Heap

Binomial Heap is a collection of Binomial trees

Binomial Tree:-

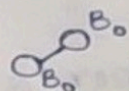
Binomial tree B_k is an ordered tree defined recursively. Binomial tree B_0 consist of single node.
A Binomial tree B_k consist of 2 binomial B_{k-1} tree that are linked together. (left biased)

$$B_k \rightarrow 2B_{k-1}$$

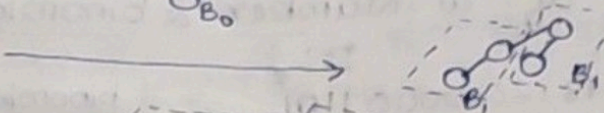


$$B_0 \rightarrow 1 \rightarrow 0$$

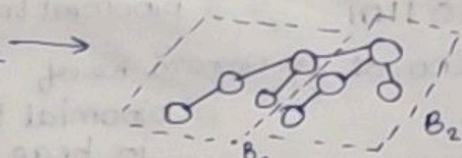
$$B_1 \rightarrow 2B_0 \rightarrow$$



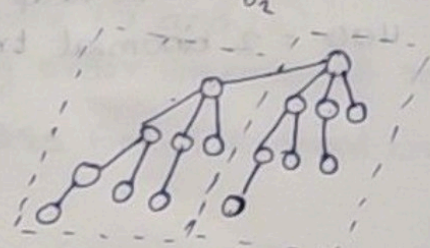
$$B_2 \rightarrow 2B_1 \rightarrow$$



$$B_3 \rightarrow 2B_2 \rightarrow$$



$$B_4 \rightarrow 2B_3 \rightarrow$$



left biased m child
add as left

Properties of binomial Tree

- 1) there are 2^k nodes.
- 2) the height of the tree is k .
- 3) there are exactly k_i nodes at depth i for exactly $i=0,1,2,3,\dots,k$.

$$k_i = \frac{k!}{i!(k-i)!} = n C_r = \frac{n!}{r!(n-r)!}$$

$$B_0 \rightarrow 0C_0 \rightarrow 1$$

$$B_1 \rightarrow 1C_0 \rightarrow 1, 1C_1 \rightarrow 1$$

$$B_2 \rightarrow 2C_0 \rightarrow 1$$

$$2C_1 \rightarrow \frac{2!}{1!} = 2$$

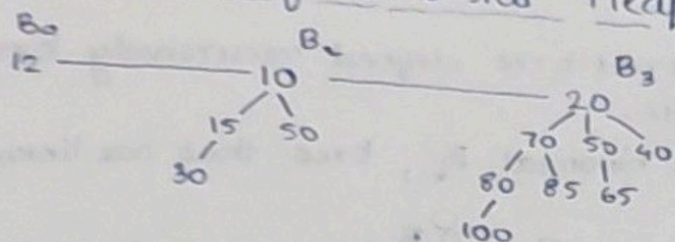
$$2C_2 \rightarrow \frac{2!}{2! \cdot (0)!} = 1$$

$$B_3 \rightarrow 3C_0 \rightarrow 1$$

$$3C_1 \rightarrow \frac{3!}{1!(2)!} = \frac{6}{2} = 3$$

4) the root has degree k and children of root are themselves a binomial tree with orders $k-1, k-2, \dots, 0$. From left to right

Representation of Binomial Heap



A binomial heap of 13 nodes is a collection of 3 binomial trees of orders 0, 2, 3

Binary Representation of a Number & Binomial Tree

Binary representation of 13 :- 0000 1101 = 3 binomial trees
 (no. of 1's correspond to no. of binomial tree in heap)

$$\begin{array}{r} 2 \mid 13 \mid 1 \\ 2 \mid 6 \mid 0 \\ 2 \mid 3 \mid 1 \\ 1 \end{array}$$

Binary representation of 12 :- 0000 1100 = 2 binomial trees

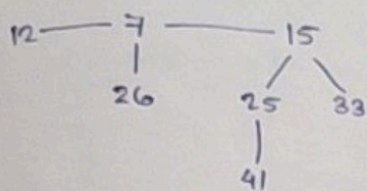
OPERATIONS

- Union (H_1, H_2)
- Insert (H, k)
- getmin (H)
- extractmin (B)
- delete (H)
- decreasekey (H)

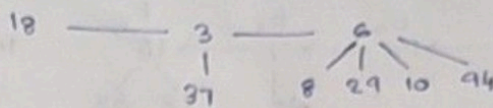
↳ Union (H_1, H_2)

- 1: First step is to simply merge the two Heaps. in non-decreasing order of keys (increasing order)
- 2: after simple merge, combine binomial trees of the same order we traverse the list of merged roots. we keep track of 3 pointers. $x, \text{next-}x, \text{next-next-}x$
 $x, n(x), n(n(x))$

H₁

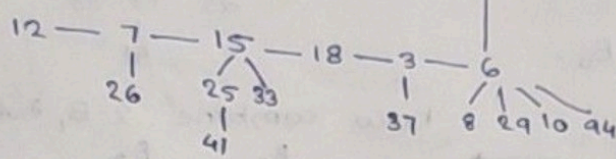


H₂

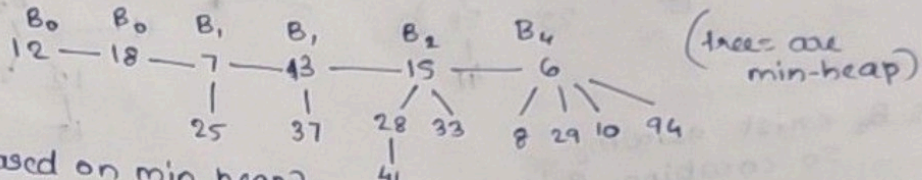


H₁ ∪ H₂

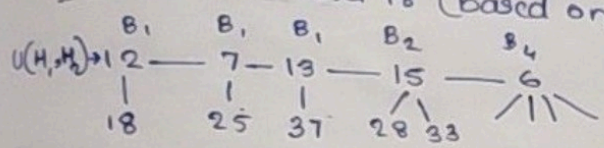
* simply merge H₁ and H₂



can be redrawn as:-



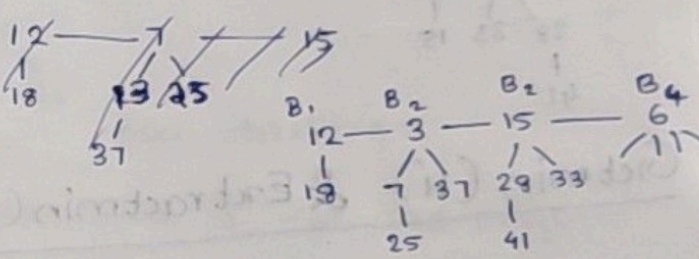
* So, combine 12 and 18 (based on min heap)



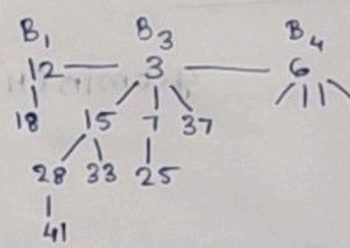
* now combine (7 and 13)

(increasing order of root values 7 and 13)

one order ~~222~~ Binomial tree structure.



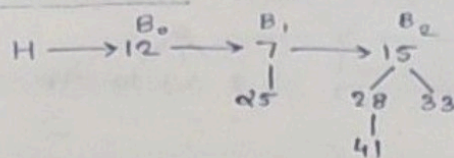
* now combine (B₂ and B₂)



the final tree after union operation having

3 binomial tree B₁, B₃, B₄

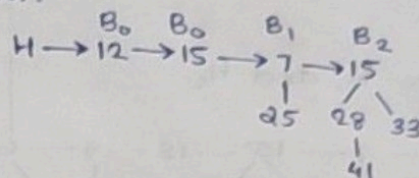
Insertion (H, k)



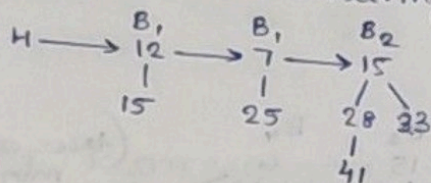
insert H → 15

and merge according to order among them.

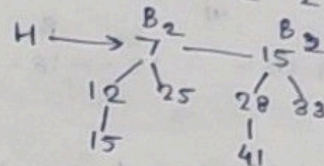
do union operation on trees:-



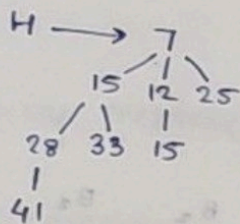
so merge 12 and 15 having B₀.



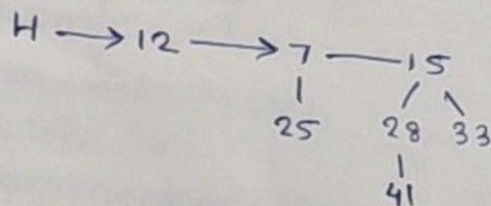
now combine 2 B₁ sub



2 B₂ exist among.
so combine B₂'s.

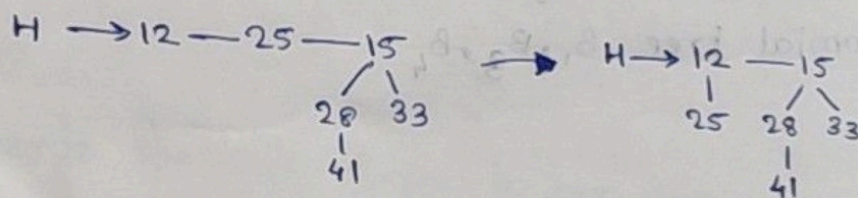


Getmin (H) & Extractmin (BH)

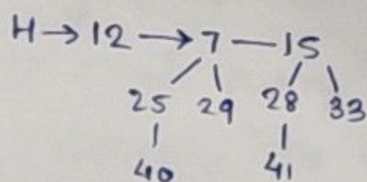


getmin(H) = 7

extractmin(H) :-



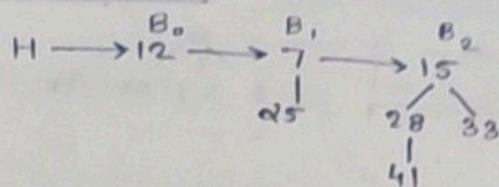
7 has 2 child 25, 29



extractmin(H) :- H → 12 → 25 → 29 → 15

2m) now merge again.

Insertion (H, k)

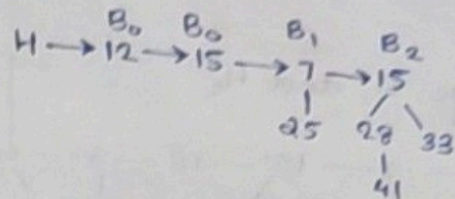
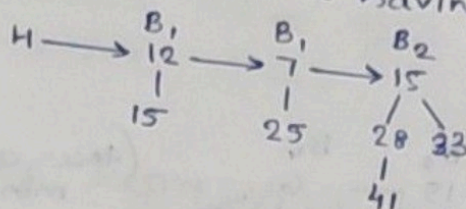


insert $H \rightarrow 15$

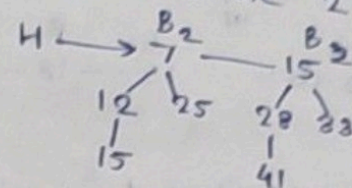
ans) arrange according to order and merge.

do union operation on trees:-

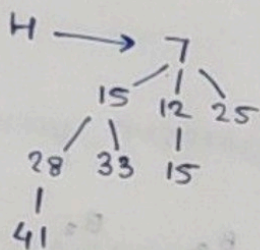
so merge 12 and 15 having B_0 .



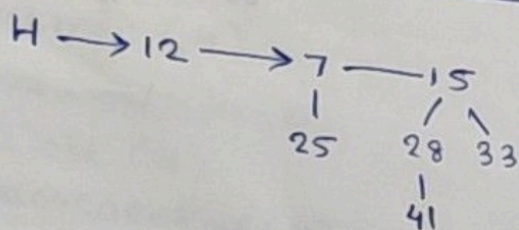
now combine 2 B_1 's.



2 B_2 exist already.
so combine B_2 's.

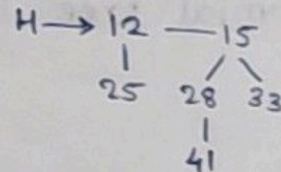
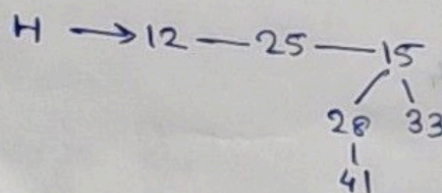


Getmin (H) & Extractmin (BH)

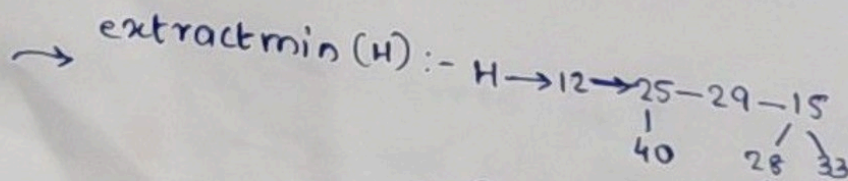
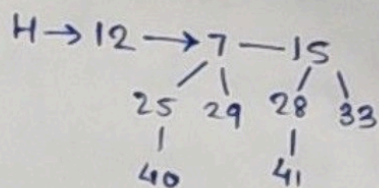


getmin(H) = 7

extractmin(H) :-



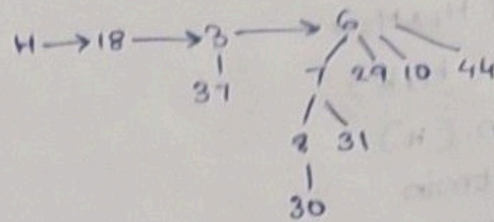
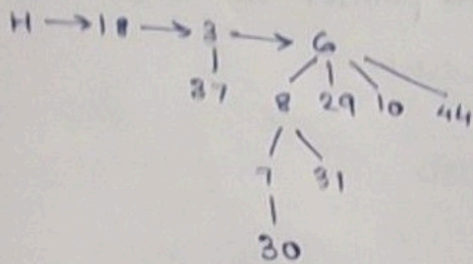
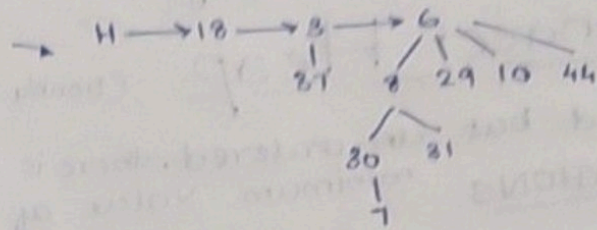
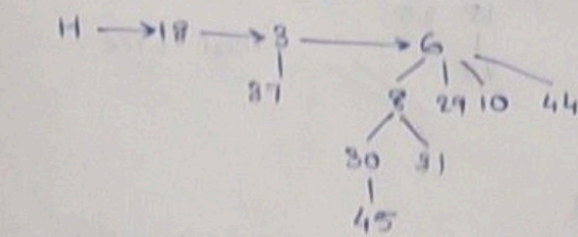
if 7 has 2 child 25, 29



2nd merge merge array.

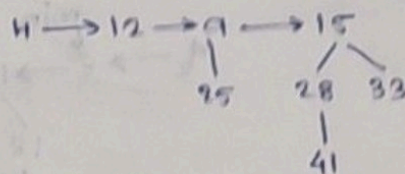
Decreasing key (10)

Decrease the key 45 by 7:-



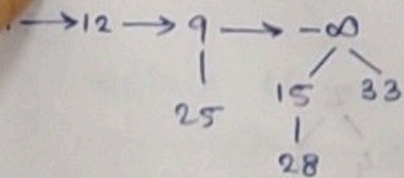
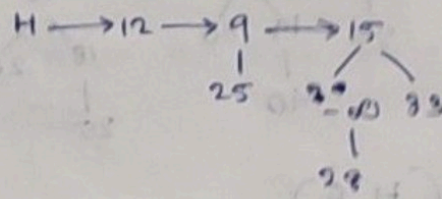
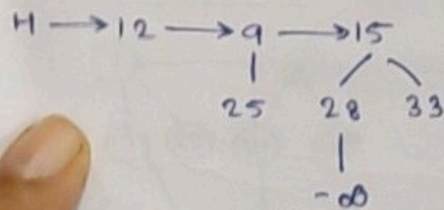
Delete (11)

Deleting a node:-



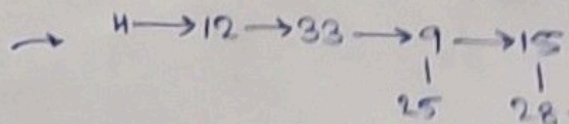
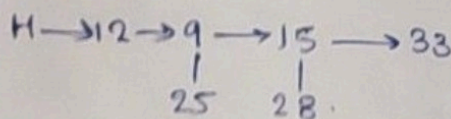
now delete 41.

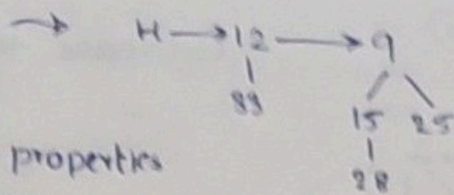
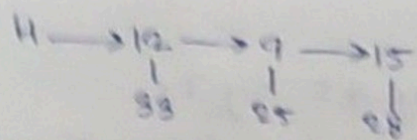
replace 41 with appropriate minimum value which is $-\infty$ and perform decreasing key operation.



$-\infty$ root list of null, so then perform extract min operation on $-\infty$. that is it is not an actual value.

doing:
after extract min:-





Final tree

Binomial Heap with max-heap properties
 getmax
 extractmax
 increasing key

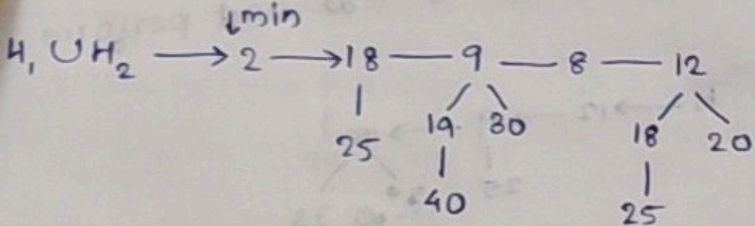
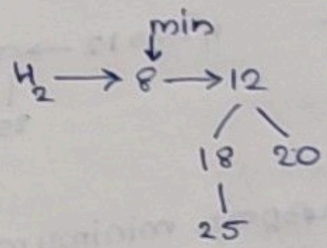
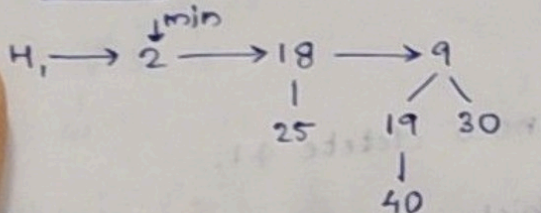
Fibonacci Heap (Doubly LL)

rooted but un-ordered. there is an extra pointer towards the minimum value of Heap.

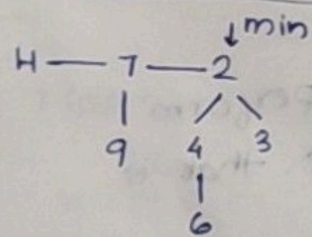
OPERATIONS

- Union (H_1, H_2)
- Insert (H, k)
- Deletion (H)
- extractmin
- Decreasekey.

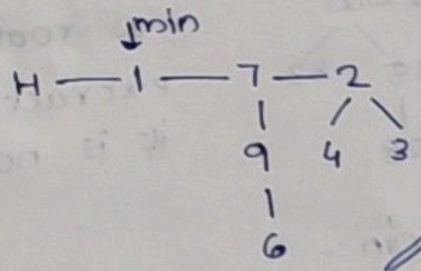
→ Union (H_1, H_2)

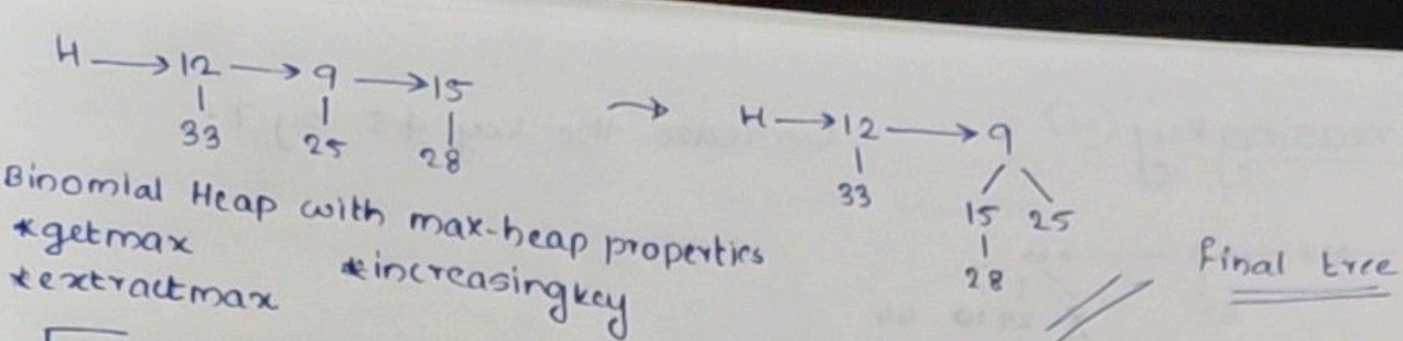


→ Insertion (H, k)



insert ①





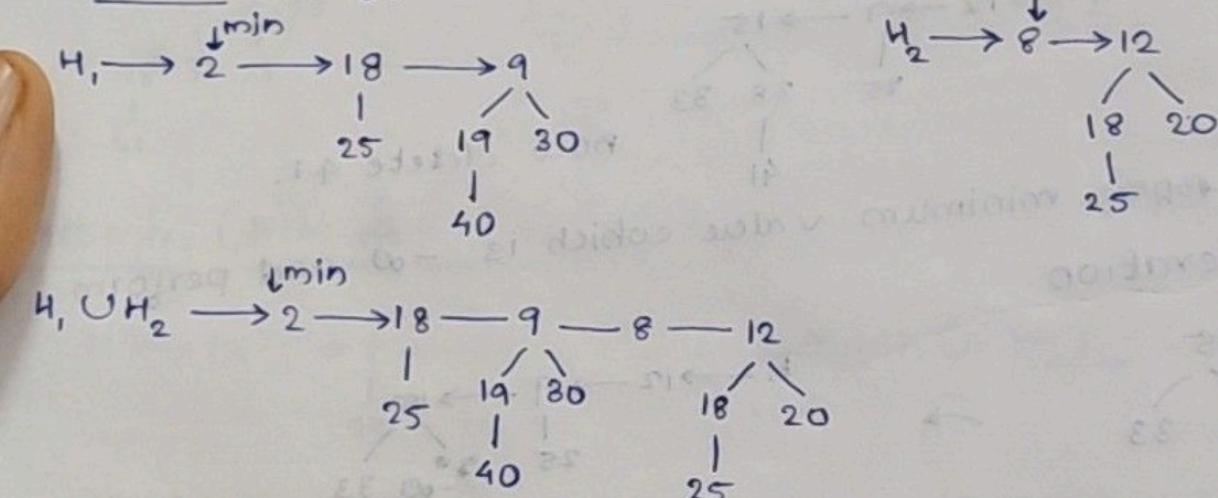
Fibonacci Heap (Doubly LL)

rooted but un-ordered. there is an extra pointer towards the minimum value of Heap.

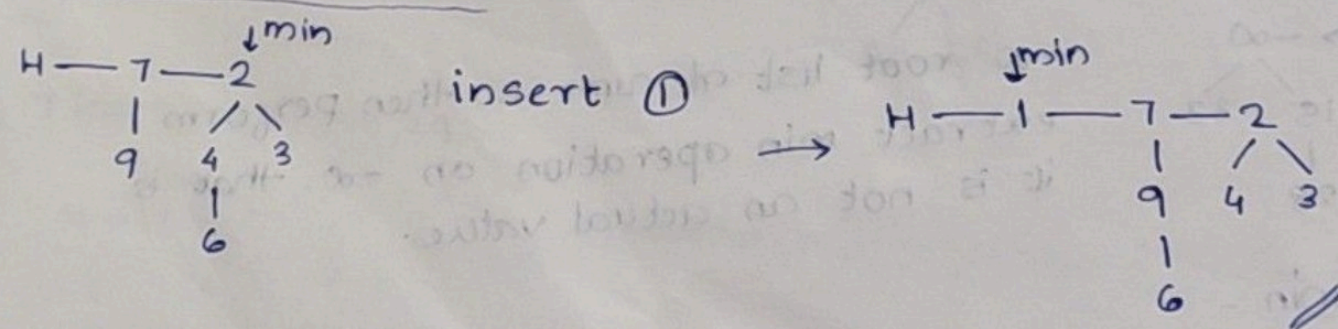
OPERATIONS

- Union (H_1, H_2)
- Insert (H, k)
- Deletion (H)
- extractmin
- Decreasekey.

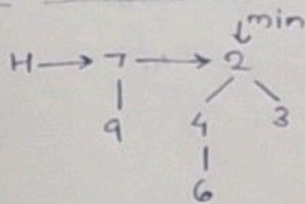
→ Union (H_1, H_2)



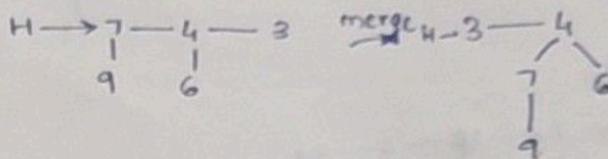
→ Insertion (H, k)



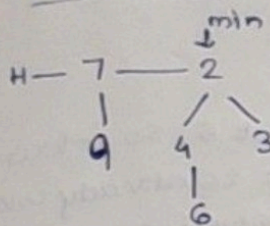
↳ Extractmin



extractingmin and insert merge 2 and 3.

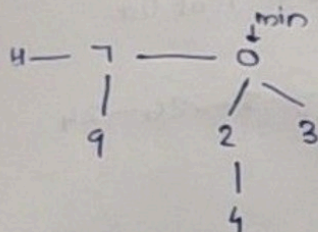
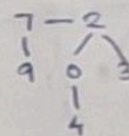
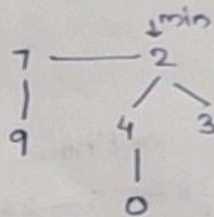


↳ Deletion

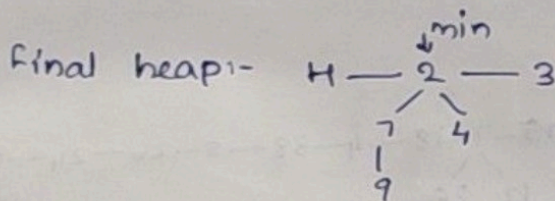
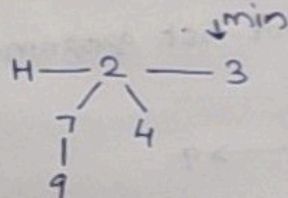
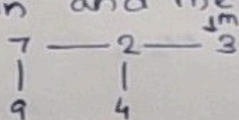


delete 6

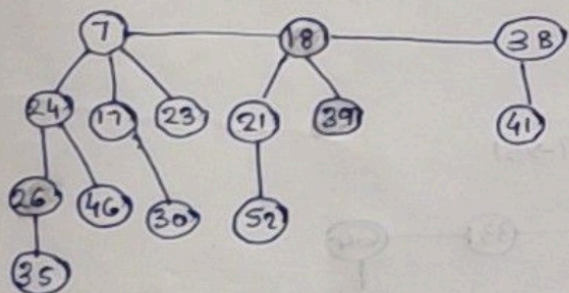
decrease 6 to 0.



now extractmin and merge



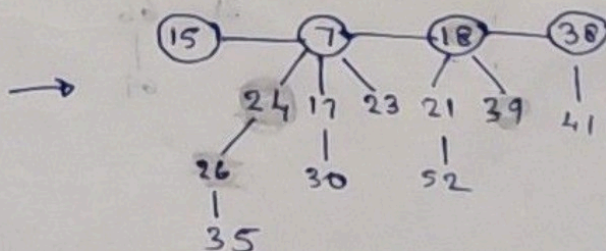
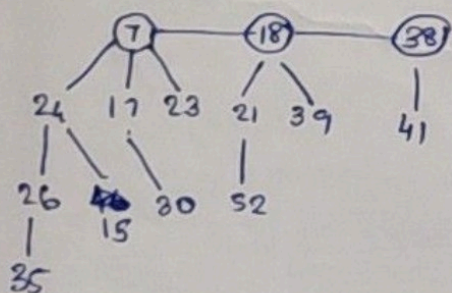
↳ Decreasekey(h)

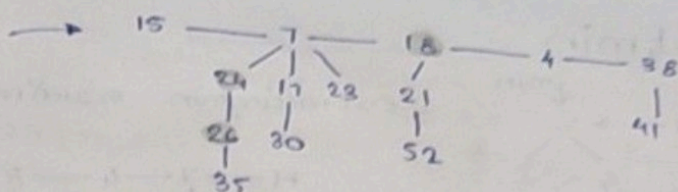
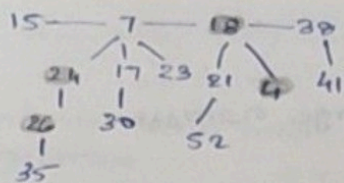


a) Decrease the key 46 to 15.

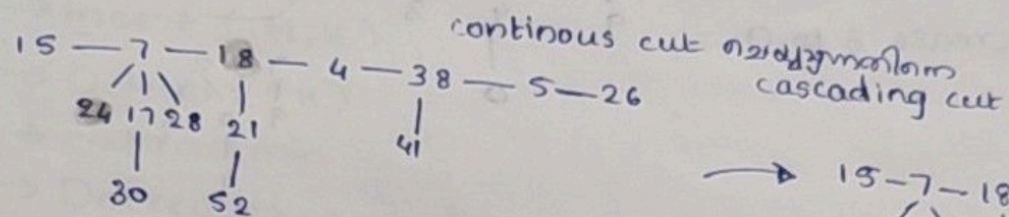
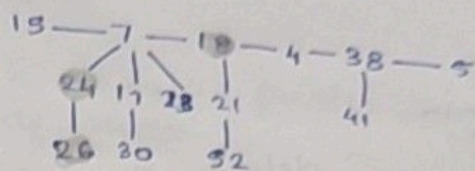
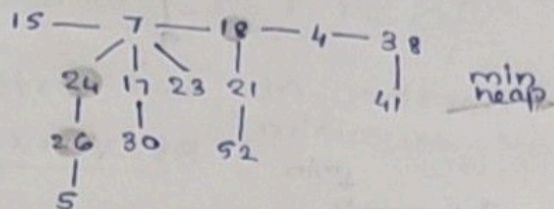
b) 39 - 4

replace 46 with 15. check whether the heap is minimum heap or not. cut the element and add to root.



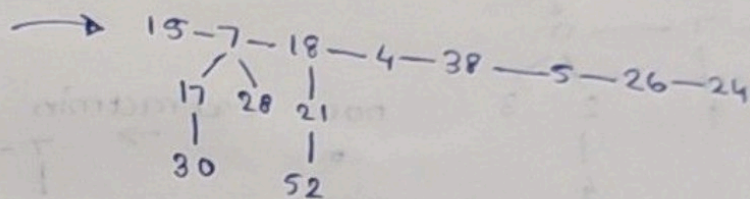


* 35 $\rightarrow$ 5

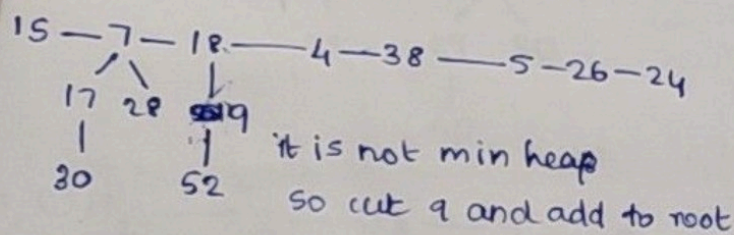


26 lost 5, so colour 26. but 26 already colour. so we need to root list

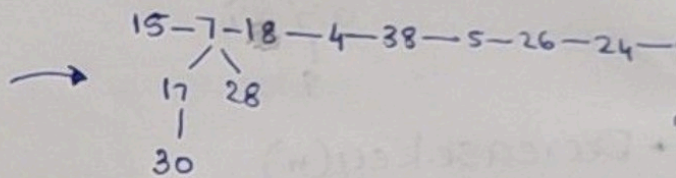
root-list 26, 5, 26, colour 26.



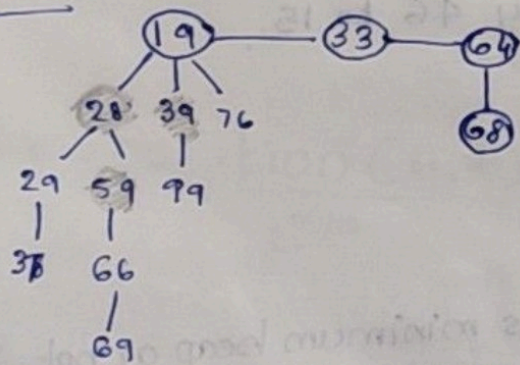
21 $\rightarrow$ 9



it is not min heap
so cut 9 and add to root



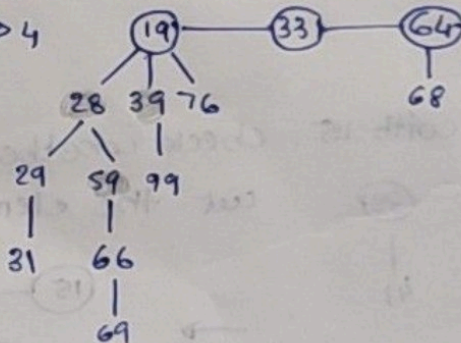
H.W



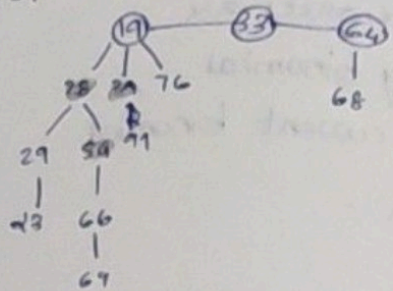
* 37 $\rightarrow$ 31

* 31 $\rightarrow$ 23

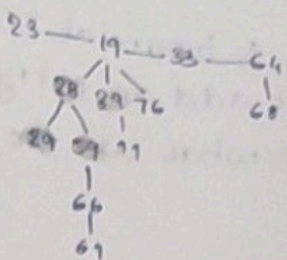
* 66 $\rightarrow$ 4



31 → 23

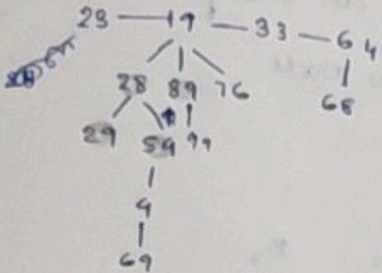


min heap

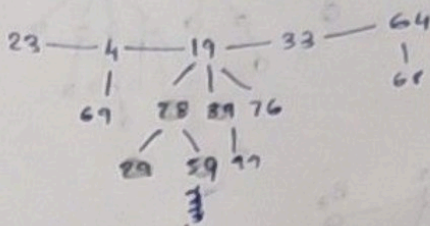


colour 29 and add 23 in root list

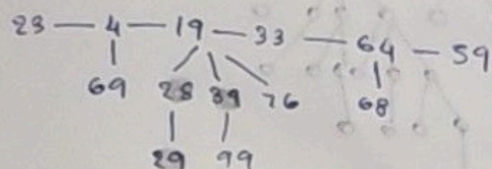
66 → 4



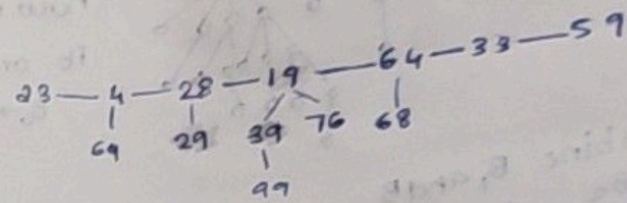
apply min heap



and colour 59 59 already colour so cut 6 add to root.



28 (parent of 59) already coloured so cut & add to root list

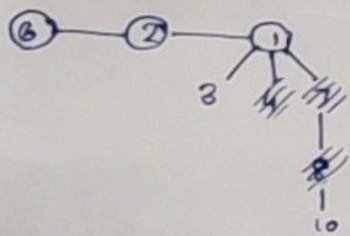


Potential of Fibonacci heap

$$\Phi(H) = t(H) + 2m(H)$$

where $t(H)$ is equal to number of trees in Fibonacci heap

$m(H)$ = number of marked nodes/coloured nodes in Fibonacci heap

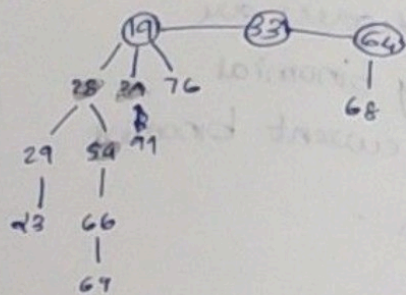


$$t(H) = 3$$

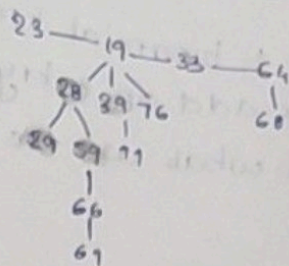
$$m(H) = 3$$

$$\therefore \Phi(H) = 3 + 2 \times 3 = 9$$

* 31 → 23

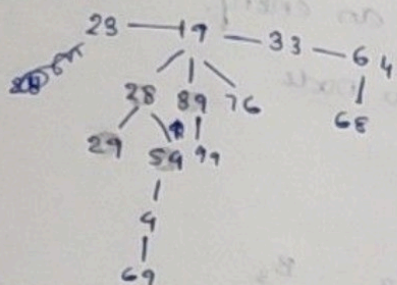


min heap

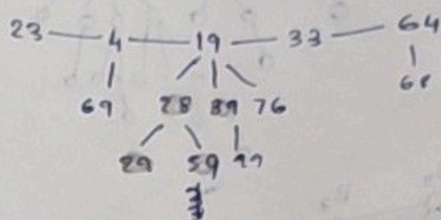


colour 29 and
cut 23
add 23 in root list

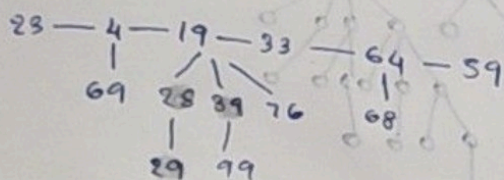
* 66 → 4



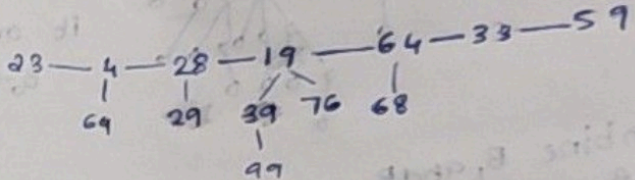
apply min heap



and colour 59
59 already colour
so cut 6 add to root



28 (parent of 59) already
coloured so cut & add to root list

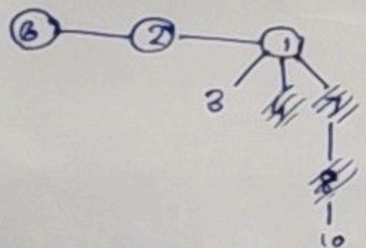


Potential of Fibonacci heap

$$\phi(H) = t(H) + 2m(H)$$

where $t(H)$ is equal to number of trees in Fibonacci heap

$m(H)$ = number of marked nodes/coloured nodes in Fibonacci heap



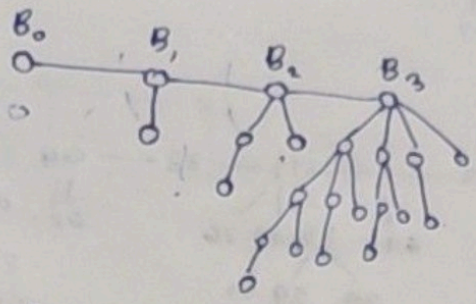
$$t(H) = 3$$

$$m(H) = 3$$

$$\therefore \phi(H) = 3 + 2 \times 3 = 9 //$$

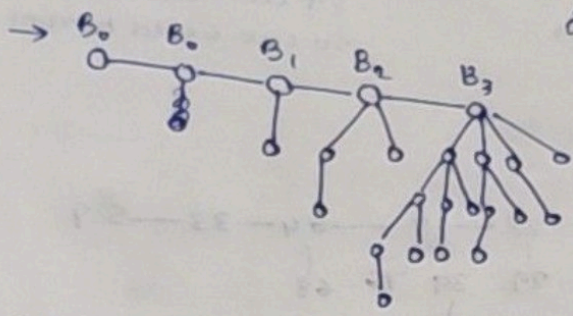
Ques: A binomial Heap has 4 binomial trees, the degrees are 0, 1, 2 and 4. after you add an entry, how many binomial trees will the heap have. what are the degrees of current binomial trees- heap.

ans)

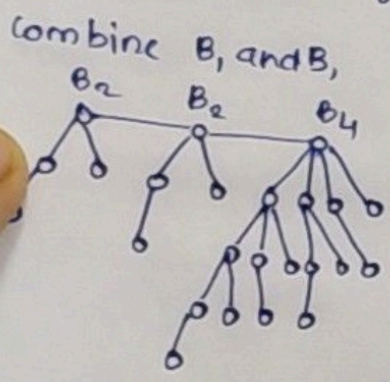
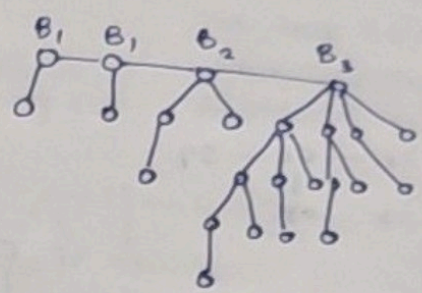


degree = order :-

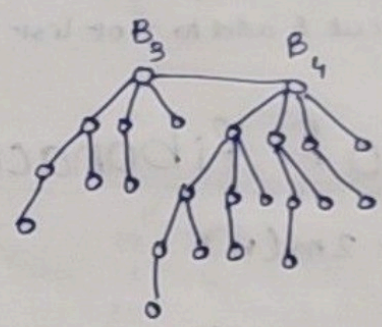
now add an entry.
∴ insert new node



now make it ordered
 $B_0 \rightarrow B_0$



now combine
 B_2 and B_2



now the degree of current binomial trees are 3 and 4. (B_3 and B_4)
higher order = 4.

draw a binomial Heap whose keys are 6, 8, 5, 18, 1, 16, 7, 9, 16, 10, 20.
 explain how union operation is performed in a binomial tree.

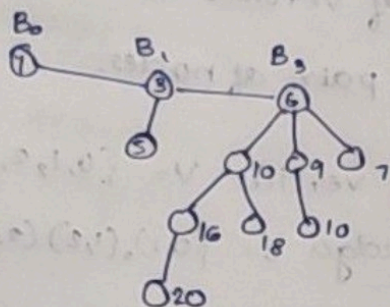
no. of keys = 11

binary rep = 1011

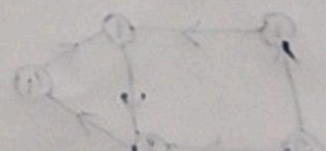
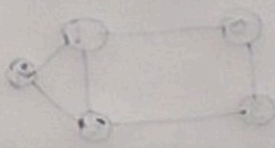
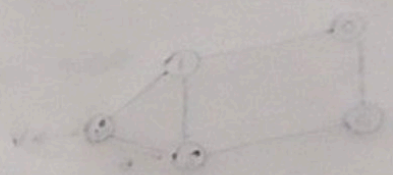
of order = 0, 1, 3

= 3 binomial trees of order

B₀



1001
1011



Question

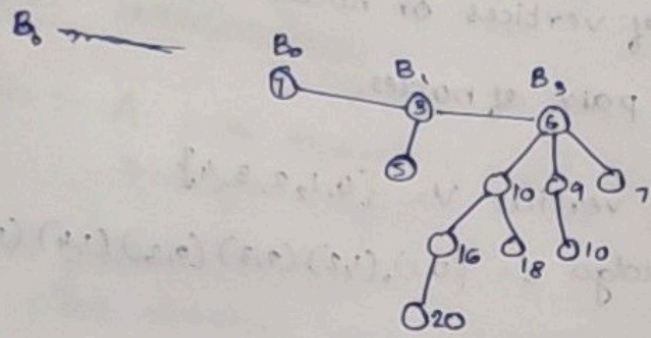
draw a binomial Heap whose keys are 6, 3, 5, 18, 1, 10, 7, 9, 16, 10, 20.
 explain how union operation is performed in a binomial tree.

no. of keys = 11

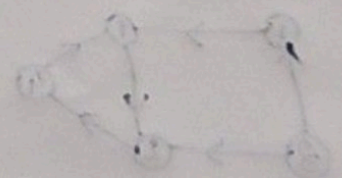
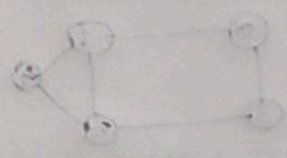
binary rep = 1011

of order = 0, 1, 3

= 3 binomial trees of order



1001
1011



Question

draw a binomial Heap whose keys are 6, 8, 5, 18, 1, 10, 7, 9, 16, 10, 20.
 explain how union operation is performed in a binomial tree.

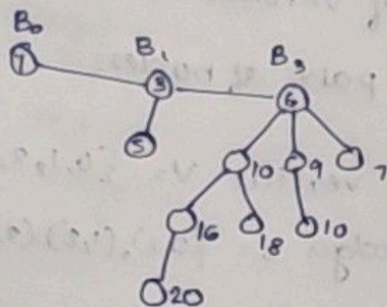
no. of keys = 11

binary rep = 1011

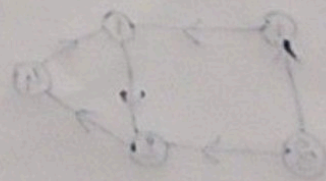
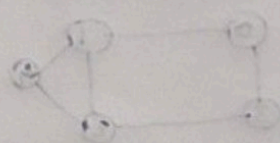
of order = 0, 1, 3

= 3 binomial trees of order

B_0



1001
1011

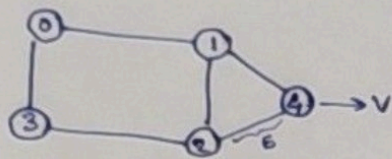


Module 4

Advanced Graph Structures.

Graph

A Graph consist of a finite set of vertices or nodes, and a set of edges / lines / Arcs which connect a pair of nodes.



set of vertices $V = \{0, 1, 2, 3, 4\}$

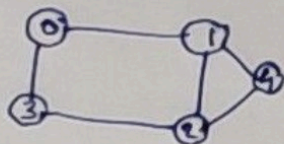
set of edges $E = \{(0,1), (1,2), (2,3), (3,0), (1,4), (2,4)\}$

Types of Graph

- 1) Directed
- 2) Undirected
- 3) Weighted

UnDirected

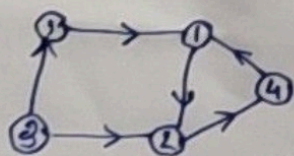
Edges are not associated with direction with them.



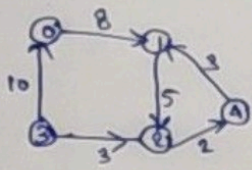
if an edge exist between 0 and 1, then the vertex can be travels from 0 to 1, as well as 1 to 0.

Directed Graph

Edges from an ordered pair edges represent a specific path from one vertex A to another vertex B. Node A is called initial node. Node B is called terminal node.



↳ Weighted Graph (Directed weighted & undirected weighted)



Each edge is assigned with some data such as length or weight. The weight of the edge e can be given a $w(e)$, which must be the value indicating the cost of traversing edge.

Graph Terminology

① Path :- A path shows a way to move from one vertex to another through edges.

② closed path :-

- The starting and ending vertex are the same.
- No other vertices repeat (except the first and last)

A-B-C-A

③ Cycle :-

- start and end vertex are same A-B-C-A
- No vertex repeats except 1st and last

④ Connected graph :-

all nodes are reachable from each other.

⑤ complete graph :-

A complete graph is a graph in which every pair of distinct vertices is connected by a unique edge. There are no missing connections.